

# Real-Time clustering and LiDAR-camera fusion on embedded platforms for self-driving cars

Micaela Verucchi\*, Luca Bartoli\*, Fabio Bagni\*, Francesco Gatti\*, Paolo Burgio\* and Marko Bertogna\*

\*University of Modena and Reggio Emilia

{micaela.verucchi, paolo.burgio, marko.bertogna}@unimore.it, {228618,228594,189382}@studenti.unimore.it

**Abstract**—3D object detection and classification are crucial tasks for perception in Autonomous Driving (AD). To promptly and correctly react to environment changes and avoid hazards, it is of paramount importance to perform those operations with high accuracy and in real-time. One of the most widely adopted strategies to improve the detection precision is to fuse information from different sensors, like e.g. cameras and LiDAR. However, sensor fusion is a computationally intensive task, that may be difficult to execute in real-time on an embedded platforms.

In this paper, we present a new approach for LiDAR and camera fusion, that can be suitable to execute within the tight timing requirements of an autonomous driving system. The proposed method is based on a new clustering algorithm developed for the LiDAR point cloud, a new technique for the alignment of the sensors, and an optimization of the Yolo-v3 neural network. The efficiency of the proposed method is validated comparing it against state-of-the-art solutions on commercial embedded platforms.

**Index Terms**—LiDAR, Clustering, Fusion, Real-Time, 3D Detection

## I. INTRODUCTION

The increasing interest on autonomous vehicles is calling for improved algorithms and techniques to execute typical self-driving tasks, including environmental perception, precise localization, and path planning. Complex road environments and the presence of several moving objects increase the complexity of these tasks. A safe autonomous driving system must rely on accurate, real-time and robust perception of the surroundings. To do so, a precise categorization and localization of “road-objects”, such as cars, pedestrians, cyclists, and other obstacles, is needed.

Currently, cameras and laser scanners (LiDARs) are the most used sensors for detection. Cameras are used to obtain textures and colors of targets, which are valuable features for classification of e.g. traffic lights and traffic signs. However, cameras have difficulties in obtaining depth information and are greatly affected by lighting conditions. On the other hand, LiDARs can more easily acquire distance and three-dimensional information, even at long range. They are robust and not affected by illumination conditions, but they lack color information, complicating the classification task. Therefore, fusing data from the two mentioned sensors has become a trend in the AD field, being a good compromise to obtain good classification and 3D detection. The main problem of this approach is its high computational cost, which prevents its adoption on real-time sensitive scenarios.

In the context of 2D camera object detection, convolutional neural networks (CNN) are often adopted. As CNNs are typically computationally expensive, a rich literature exists targeting faster models with reasonable performance. Yolo [1], SSD [2], MobileNet [3], FastRCNN [4] and CenterNet [5] offer the best compromise between timing and accuracy performance.

LiDARs instead produce a 3D point cloud, which is then processed to place 3D bounding boxes (BBs) around the objects via clustering methods. Several types of clustering techniques exist [6] [7]: centroid-based, distribution-based, hierarchical and density-based clustering. This latter category is the most suitable for processing LiDAR point clouds. The most widely adopted algorithm in this class is DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [8].

Unfortunately, merging camera detection with LiDAR processing makes it very challenging to meet the real-time requirements of the perception tasks even on latest generation embedded computing platforms, like NVIDIA AGX Xavier. To address this problem, this work will propose:

- a clustering technique inspired by the DBSCAN algorithm to better exploit the features of a LiDAR point cloud;
- a new method for camera and LiDAR alignment;
- an optimized version of Yolo-v3 [9] for NVIDIA embedded boards;
- an open-source real-time fusion framework that combines camera 2D BBs with LiDAR clusters.

The paper is organized as follows: section II provides an overview of the state-of-the-art (SoA) for clustering and fusion. Section III describes the Window Based LiDAR Clustering (WBLC) algorithm. Section IV introduces our system model, focusing on calibration, alignment of the sensors and fusion. Finally, the proposed approach is evaluated in V, comparing it against different existing methods.

## II. RELATED WORKS

Our integrated approach merges different techniques, including LiDAR clustering, object detection, sensor fusion and calibration. We hereafter detail the state-of-the-art techniques on each of these topics, with a specific focus on embedded systems, explaining how our work builds upon them.

### A. LiDAR clustering

LiDAR sensors perceive the external world through 360° laser scans, producing a point cloud of the environment to detect surrounding objects. A natural way to do that is via clustering. One of the most widely adopted technique for clustering is DBSCAN [8]. The DBSCAN algorithm discovers clusters of arbitrary shapes and is efficient even for large spatial databases. The algorithm seeks for clusters by searching the neighborhood of each object: if two objects satisfy some density criteria, they are correlated. If the number of correlated objects is greater than a user-defined threshold, a cluster is formed. Unlike other techniques, such as K-Means [10], DBSCAN can classify points as noise, discarding them from the real clusters.

The complexity of DBSCAN is  $\mathcal{O}(n^2)$  in the number  $n$  of points. Since a LiDAR point cloud is composed of tens of thousand points, it is very difficult to fit the clustering task within the required real-time latency. A smaller complexity of  $\mathcal{O}(n \log n)$  can be achieved using a kd-tree-based DBSCAN. Still, constructing a kd-tree is quite complex and time-intensive. In the literature, several GPU implementations of the original algorithm can be found (e.g. [11], [12], etc.), which however are not able to achieve the required real-time performance for the automotive platform of interest.

Many other methods have been presented to improve real-time performance of clustering tasks, like [13], [14], [15], [16]. They all trade performance for precision in the clustering method. One of the most closely related works with the approach proposed in this paper has been presented by Wen et al. in [17], applying DBSCAN to LiDAR point clouds for improving the performance by (i) removing the ground points and (ii) bounding the search of the neighbors for each object to a fixed number  $k$ . However, a 2D depth map has first to be generated from the 3D point cloud, with a non-negligible computational overhead.

In this work, a smarter method is presented for real-time clustering, customizing the DBSCAN algorithm for LiDAR point clouds, without losing accuracy on cluster detection.

### B. LiDAR-Camera fusion and calibration

Once clusters are found on the LiDAR cloud, and BBs are obtained on camera frames using a real-time CNN, the challenge is fusing the information. One approach for camera and LiDAR fusion is by using extrinsic calibration. Most calibration approaches are based on identifying and matching features acquired from the two sensors and determining the calibration parameters. The most common approaches require the existence of calibration patterns in the scene, i.e. checkerboards [18], polygonal boards [19], circles targets [20] and so on. When calibrating LiDAR and cameras, good results can be achieved with a perforated panel with four circles [21]. This pattern, introduced by Velas et al. in [22], is effective because circles can be easily found both from camera frames (using Sobel filter [23] and Hough transforms [24]) and from LiDAR point cloud (thanks to RANSAC methods [25]). Once the centers of the circles are found, the alignment can be easily

performed. LiDAR points can then be projected on a plane and aligned with camera frames.

Different works exist to improve the detection accuracy of well-known CNNs by using LiDAR data. In [26], Wang et al. improve Yolo-v3 for detecting 2D BBs. LiDAR point cloud clusters are exploited to propose a region of interest (ROI) on the camera frame, where Yolo-v3 is selectively applied to improve accuracy.

To estimate 3D BBs, different works can be found in the literature for camera only [27] [28], for LiDAR [29] [30], or fusing both [31] [32]. Among these latter ones, Qi et al. present in [33] an approach based on frustum pointnets for 3D object detection. Starting from a pre-aligned depth image, they propose a framework composed of three steps: frustum proposal, 3D instance segmentation, and 3D box estimation. They make use of several neural networks (NN), achieving a high accuracy, but paying a high latency.

A better real-time performance is achieved by Wei et al. in [34], performing sensor fusion at 5Hz on an NVIDIA Tx2 board. In their setting, they use Yolo-v2 [35] to estimate BBs and a second NN to project them in 3D coordinates. For the LiDAR output, they apply a modified DBSCAN algorithm and train an SVM classifier. The BBs from both camera and LiDAR are then fused to provide the final output. The main limitation is that the detection focuses only on street beacons.

In [36], Ali et. al. customized Yolo-v2, proposing Yolo3D, a real-time 3D oriented BB detector for LiDAR point clouds. Detection and classification is performed only on LiDAR data, constructing two bird's eye view grids: a height grid and a density grid. Yolo-v2 is then modified to predict 8 parameters: 3D BB center ( $x, y, z$ ), 3D dimensions (length, width, height), orientation in bird-view space and confidence.

In this paper, we present a real-time fusion framework, similar to [34], that is not limited to beacons, but it is suitable for every "road-object". A DBSCAN improvement similar to [17] is applied, but with significantly faster results. A calibration technique similar to [22] is adopted, but with a different projection method. Overall, better performance in accuracy and time is achieved, being able to predict 3D BBs in 29 ms on an embedded platform.

## III. WINDOW-BASED LIDAR CLUSTERING

To improve existing LiDAR clustering methods for achieving real-time performance on the considered embedded computing platforms, three expedients have been adopted:

- light cloud preprocessing;
- parallelization; and
- limited neighbour exploration.

We hereafter explain each of these steps.

### A. Preprocessing

The preprocessing task aims at removing from the sampled LiDAR point cloud all points corresponding to the ground or higher than the expected height of the objects to be detected. This allows significantly decreasing the computational complexity of later clustering steps.

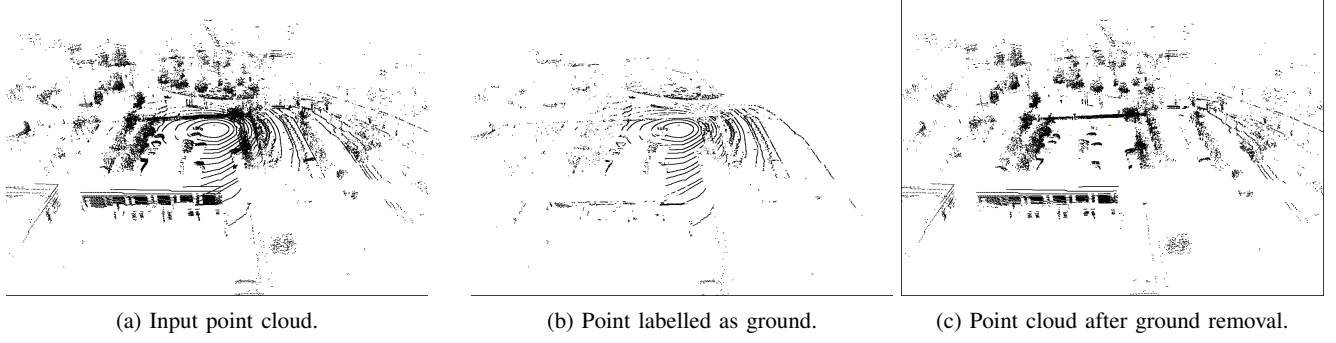


Fig. 1: Ground removal in a LiDAR point cloud.

The proposed technique works by considering all neighbouring points in order, classifying as ground all points that are within a given vertical distance and angle from the previous one, propagating in this way the notion of ground along the point cloud. This method is more robust to slopes than a method that simply removes all points below a given absolute value.

More in detail, one origin point  $p_O$  is created in a ground position as close as possible to the car. In our case, the selected point  $p_O$  was  $(0, 0, -1.80)$ , since the height of the roof of the car where the LiDAR was mounted is 1.8m. Then, LiDAR layers are sorted according to the corresponding sampling angle. For each point in each layer, one thread is spawned. Initially, the layer with the smallest angle (layer 0) is chosen. Each point  $p_i^0$  belonging to this layer is compared with the origin point. The angle between  $p_O$  and  $p_i^0$  is computed, as well as their height difference. If those values are less than a user-defined threshold,  $p_i^0$  is labelled as ground. If so, the corresponding point on the next layer  $p_i^1$  is compared with  $p_i^0$ ; otherwise, it is compared with  $p_O$ , and so on. Once all the points have been checked, the ones labeled as ground can be removed. Figure 1 shows an example of ground removal obtained with this technique.

### B. Clustering

The clustering algorithm is performed only on two dimensions, i.e. latitude and longitude, dropping height. This choice follows by assuming objects are never placed one on top of another.

The search of the neighboring points is then parallelized on a GPU, creating a thread for each point. Moreover, the exploration is bounded to a user-defined threshold, checking at most  $2W$  points for each object in the cloud. This is a reasonable choice for a LiDAR sensor because of the nature of the output data, where cloud points are semi-ordered by horizontal angle. Exploiting this feature, the search for neighbouring points can be limited to adjacent angles.

The proposed Window-Based Lidar Clustering (WBLC) is detailed in Algorithm 1, receiving a 2D point cloud as input, and producing the identified clusters as output. The algorithm is divided into two parts: the searching for neighbors and the merging of clusters.

---

#### Algorithm 1: WBLC

---

**Input:** pointcloud  $\mathcal{P}$ ,  $W$ ,  $maxN$ ,  $maxD$ ,  $minCluster$

**Output:** *clusters*

**Data:**  $nList(p_j) = \emptyset \forall p_j \in \mathcal{P}$ ,  $clusters = \emptyset$ ,  $dir = next$

```

1 forall  $p_j \in \mathcal{P}$  do
2   for  $i = 0$  to  $2W$  do
3     if  $i == W$  then
4        $dir = prev$ ;
5      $p_k = nextNeigh(\mathcal{P}, p_j, i, dir)$ ;
6      $d = euclideanDistance(p_j, p_k)$ ;
7     if  $d < maxD$  then
8        $nList(p_j) = nList(p_j) \cup p_k$ ;
9       if  $i < W$  and  $len(nList(p_j)) > \lfloor \frac{maxN}{2} \rfloor$ 
10        then
11           $i = W$ ;
12        if  $i \geq W$  and  $len(nList(p_j)) > maxN$ 
13          then
14            break;
15 forall  $p_j \in \mathcal{P}$  do
16    $\mathcal{P} = \mathcal{P} \setminus p_j$ ;
17   forall  $p_i \in nList(p_j)$  do
18      $\mathcal{P} = \mathcal{P} \setminus p_i$ ;
19     forall  $p_k \in nList(p_i)$  and  $p_k \notin nList(p_j)$  do
20        $nList(p_j).enqueue(p_k)$ ;
21   if  $len(nList(p_j)) > minCluster$  then
22      $clusters = clusters \cup nList(p_j)$ ;
23 return clusters;

```

---

In the former part (lines 1-12), a window of size  $2W$  is explored for each point  $p_j$  in the point cloud  $\mathcal{P}$ , checking  $W$  preceding and  $W$  following points. Points that meet a distance criterion are identified as neighbours and stored in a list  $nList(p_j)$ . Once  $maxN$  neighbors have been found, the search in the window is stopped to limit memory usage and

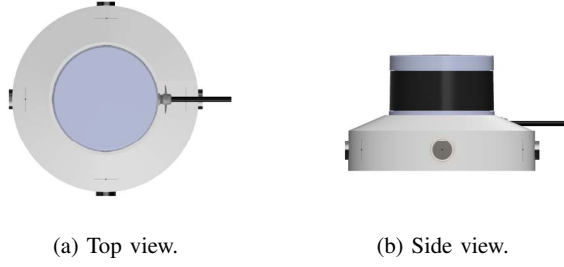


Fig. 2: Support for cameras and LiDAR.

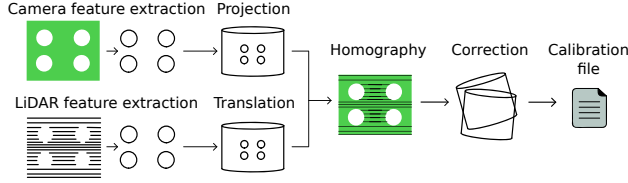


Fig. 3: Calibration scheme.

computational complexity.

In the latter part (lines 13-20), the neighbors list of each point is iteratively inspected (lines 17-18) to merge all the lists that belong to the same clusters. The *if* statement at line 19 checks whether a list of neighbors contains more than a minimum number of points *minCluster*. Clusters below this threshold are considered as noise, and not included in the identified clusters returned at line 21.

To evaluate the complexity of the algorithm, consider that, in the first part, at most  $2W$  points are checked for each of the  $|P|$  points, leading to a complexity of  $O(W|P|)$ . In the second part, at most  $maxN$  points are checked for each of the  $|P|$  points, leading to  $O(maxN|P|)$ . Since  $maxN$  is selected to be smaller than  $W$ , the overall complexity is  $O(W|P|)$ .

#### IV. FUSION

##### A. System Model

The self-driving prototype used for this work mounts a Velodyne ULTRA Puck VLP-32C LiDAR and 4 Sekonix cameras SF332X-10X with  $120^\circ$  FOV mounted on the support shown in Fig. 2. The LiDAR is located in the center of the support, surrounded by the four cameras covering  $360^\circ$ . The support is then placed on the roof of the car. For sensors calibration, a green flat panel with four holes has been adopted. The adopted computing platform is an NVIDIA AGX Xavier, a recently released embedded platform featuring a Tegra System on Chip (Xavier SoC) composed of 8 NVIDIA Carmel CPU Cores (superscalar architecture compatible with ARMv8.2 ISA) and an integrated GPU based on the NVIDIA Volta architecture with 512 CUDA cores.

##### B. Calibration

For the considered setting, the proposed calibration scheme is depicted in Fig. 3. It consists in five phases following the initial intrinsic calibration of the camera.

- 1) *Feature extraction*. The calibration pattern is registered both from camera and LiDAR. From the camera frame, the pattern is extracted via a green color filter and, thanks to Hough transform for circles [24], the center of the holes is easily computed. From the LiDAR, a range filter is used to find the panel, a Sobel filter [23] to extract the edges, and a conic fitting [37] to select the centers.
- 2) *Cylindrical projection*. The camera calibration matrix is cylindrically projected to obtain a coherent representation with the LiDAR.
- 3) *Cylinders alignment*. The LiDAR cylinder is shifted of a fixed offset (in cm) given by the positioning of the sensors. The camera cylinder is aligned with the LiDAR one by homography, exploiting the centers of the holes found at step 1.
- 4) *Evaluation*. A good calibration is achieved when the vertical edges of the distorted camera frame, projected on the LiDAR depth map using the calculated homography, are parallel and have the same length. If this is not the case, a further correction is needed (step 5). Otherwise, calibration is complete.
- 5) *Correction*. Translation and rotation can be corrected as follows:

- Translation: define a neighborhood of the sensors positioning displacement to adjust the overlapping, moving the LiDAR cylinder. Go back to step 4.
- Rotation (roll): from the camera frame pick two points that, projected on the LiDAR cylinder, have the same *y* value. Check their angle on the camera image w.r.t. the *x*-axis: if the angle is not zero, the camera image should be rotated by that angle. Go back to step 2.
- Rotation (pitch): the perpendicular point of the camera cylinder needs to match the LiDAR one. If these two points have a different height when projected to the horizon, the camera distortion must be fixed. To do so, replace the focal center of the camera with the point that is aligned with the LiDAR one when projected to the cylinder. Go back to step 2.

##### C. Camera detection

From camera frames, objects are detected and classified with an optimized version of Yolo-v3. To improve performance, our implementation has been ported to NVIDIA TensorRT<sup>1</sup>. The source code is provided to be freely downloaded from GitHub<sup>2</sup>. Moreover, a training from scratch has been performed on the BDD100K Berkeley Dataset [38] (with 10 classes) to better recognize road objects.

##### D. Fusion

Once objects are detected both in the frame and in the point cloud, fusion is performed as depicted in Fig. 4. Camera frame BBs and clusters given by WBLC are projected thanks

<sup>1</sup><https://developer.nvidia.com/tensorrt>

<sup>2</sup><https://github.com/ceccocats/tkDNN>

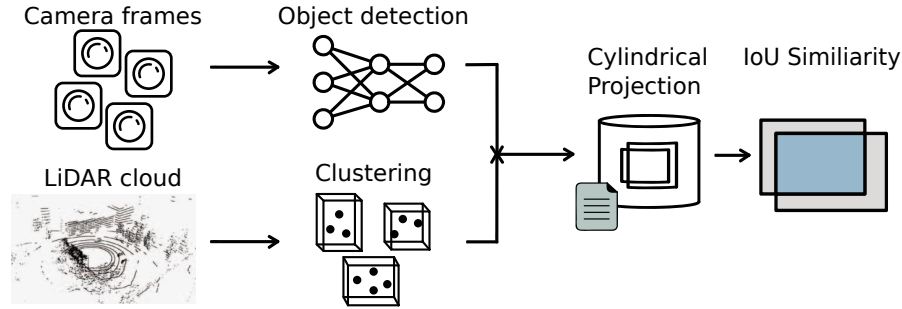


Fig. 4: Fusion scheme.

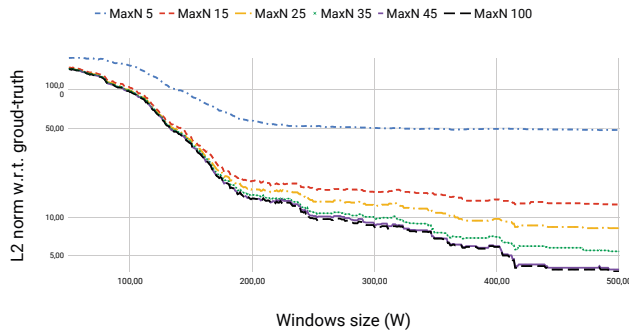


Fig. 5: Clustering error varying  $W$  and  $maxN$ .

to the precomputed calibration file, and sequentially merged. The Intersection over Union (IoU) metric is adopted to check if a BB and a cluster match the same object. If the IoU is greater than a user-defined threshold, they are deemed to represent the same object. In this case, clusters are labeled using the corresponding BDD100K class (e.g., pedestrian, car, bus, traffic light, etc). Clusters that do not match any BB (e.g., trees, bins, etc.) are labeled with the “unknown” class.

The code for clustering and fusion, as well as the adopted dataset, are released open source and available to download<sup>3</sup>.

## V. EXPERIMENTS

In this section, we perform an exhaustive evaluation of the proposed techniques, comparing them against the state-of-the-art.

### A. Clustering

To validate the clustering algorithm, several experiments have been carried out. We hereafter first show how the selection of the parameters  $maxN$  and  $W$  affect the clustering performance, considering both time and accuracy.

We created a dataset of recordings taken in a real city with the sensor set described in Section IV. The ground truth for cluster identification was given by DBSCAN, as it provides the best precision and recall, although at a very high computational complexity. The clustering error of our algorithm varying  $W$

<sup>3</sup><https://github.com/lucabart97/WBLC>, [https://git.hipert.unimore.it/fbagni/kitti\\_fusion](https://git.hipert.unimore.it/fbagni/kitti_fusion)

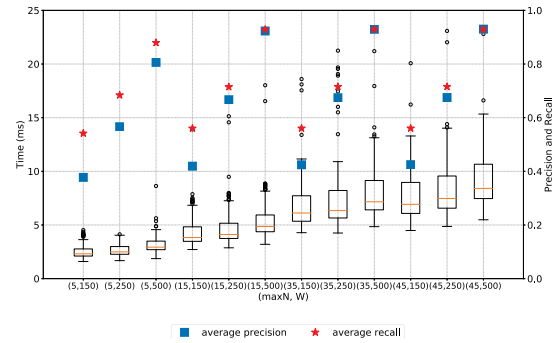


Fig. 6: Execution time on Xavier (box-plots), precision (squared dot) and recall (starred dot) of the clustering algorithm.

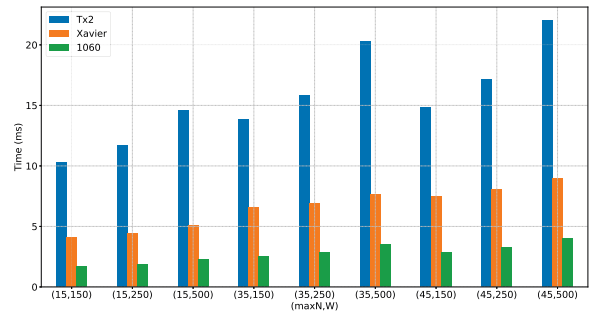


Fig. 7: Performance comparison of average time between NVIDIA TX2, Xavier and GTX1060.

and  $maxN$  is shown in Fig. 5. For every combination of  $maxN$  and  $W$ , 25 point clouds have been tested. The y-axis shows, in logarithmic scale, the euclidean distance (L2 norm) between the vector of detected clusters obtained with our algorithm and that obtained with DBSCAN. The smaller the value, the better the precision. This metric was used for a preliminary evaluation of the considered combinations. As can be seen in the chart, higher values of  $W$  lead to a more precise clustering, and there is no substantial improvement for  $maxN > 45$ . Therefore, for the following tests, we picked only the 12 most representative combinations of  $W = 150, 250, 500$  and  $maxN = 5, 15, 35, 45$ .

TABLE I: Clustering timing performances comparison.

| Method                  | Platform                                         | Dataset                    | Time (ms)                         |
|-------------------------|--------------------------------------------------|----------------------------|-----------------------------------|
| DBSCAN [17]             | i7-6700 @3.4GHz, GTX 1060, Velodyne HDL-32E      | N/D                        | 26411.6                           |
| kd-tree DBSCAN [17]     | i7-6700 @3.4GHz, GTX 1060, Velodyne HDL-32E      | N/D                        | 779.142                           |
| Range based DBSCAN [17] | i7-6700 @3.4GHz, GTX 1060, Velodyne HDL-32E      | N/D                        | 46.14                             |
| FRIBS [13] 32-beams     | i7 4770K @3.5 GHz, Velodyne Puck VLP-16          | 2 500 clouds (15 000 pts)  | $2.6 \pm 0.5$                     |
| FMP3DPSB [15]           | i7-4790K @4GHz, Velodyne HDL-64E S2              | 3 000 clouds (130 000 pts) | 15                                |
| FPCL [14]               | i7-4790K @4GHz, Velodyne HDL-64E S2              | 4000 clouds (130 000 pts)  | 36                                |
| FDDMGC [16]             | i7-6700 @3.4 GHz, Velodyne HDL-32E               | 1 cloud (60 000 pts)       | 11.01                             |
| this paper              | Tegra Jetson TX2, Velodyne VLP-32C               | 2 963 clouds (60 000 pts)  | $14.60 \pm 2.75$                  |
| this paper              | AGX Xavier, Velodyne VLP-32C                     | 2 963 clouds (60 000 pts)  | $5.11 \pm 0.98$                   |
| this paper              | i7-7700 @3.60GHz, GTX 1060-6GB, Velodyne VLP-32C | 2 963 clouds (60 000 pts)  | <b><math>2.29 \pm 0.43</math></b> |

TABLE II: Clustering accuracy performances.

| Method              | Dataset         | Precision | Recall |
|---------------------|-----------------|-----------|--------|
| FRIBS [13] 64-beams | Dataset by [16] | 0.69      | 0.71   |
| FMP3DPSB [15]       | Dataset by [16] | 0.87      | 0.84   |
| FDDMGC [16]         | Dataset by [16] | 0.94      | 0.96   |
| this paper          | Our Dataset     | 0.923     | 0.928  |

For each considered combination, Fig. 6 shows a more detailed characterization of the performance, reporting the average precision (red stars) and recall (blue square) of our algorithm, based on IoU of the identified clusters on 1000 point clouds. Selecting  $W = 500$  allows obtaining almost the same performance as the reference DBSCAN algorithm, with a limited dependency over  $maxN$ . The figure also reports the execution time of our algorithm on the NVIDIA AGX Xavier platform. Each boxplot provides the distribution of the execution time in milliseconds, with the central line representing the average, and the boundaries representing maximum and minimum time. As expected, the execution time grows increasing  $maxN$  and  $W$ .

Similar results have been obtained also when executing on other platforms, as shown in Fig. 7, representing the average execution time of the proposed algorithm on (i) NVIDIA Tegra Tx2, (ii) NVIDIA AGX Xavier, and (iii) a desktop with Intel CPU i7-7700 at 3.60GHz featuring a discrete NVIDIA GPU GTX 1060-6GB.

Considering the presented results, selecting  $W = 500$  and  $maxN = 15$  offers the best trade-off between execution time and accuracy. We will therefore select these values for our algorithm, and compare it against state-of-the-art solutions.

Table I compares the performance of our clustering method against all the techniques introduced in Section II. The proposed algorithm outperforms all the other methods on general purpose desktops and discrete GPUs, and it is the only one that is able to provide real-time performance even on embedded platforms (TX2 and Xavier).

Notably, these results are obtained without sacrificing accuracy. Table II details precision and recall of the considered algorithms. Since the dataset adopted in [16] is not publicly available, our urban dataset was used as a comparable benchmark for the proposed clustering method. Our algorithm has a significantly higher accuracy than the fastest competing

method (FRIBS [13]). It also has a comparable accuracy to the most precise method (FDDMGC [16]), which however runs five time slower.

TABLE III: Calibration performances, miscalibration error.

| Method                         | Miscalibration Error $E_P$ [%] |
|--------------------------------|--------------------------------|
| This paper, calibration frames | 6                              |
| This paper, other frames       | 7                              |
| Velas [22]                     | 14                             |
| LT [39]                        | 18                             |

TABLE IV: Performance comparison of Yolo neural networks on different frameworks.

| Framework         | Yolov2 (fps) | Yolov2-tiny (fps) | Yolov3 (fps) |
|-------------------|--------------|-------------------|--------------|
| darknet [1]       | 3,39         | 26,91             | 1,63         |
| deepstream (FP16) | 6,88         | 35,98             | 4,41         |
| tkDNN (FP32)      | 6,91         | 50,78             | 5,61         |
| tkDNN (FP16)      | <b>10,02</b> | <b>67,06</b>      | <b>7,44</b>  |

TABLE V: Execution time of the fusion framework on different platforms.

| Time(ms) | Tx2    | AGX Xavier | GTX 1060-6GB | Titan V |
|----------|--------|------------|--------------|---------|
| AVG      | 147,80 | 29,43      | 28,54        | 6.66    |
| MIN      | 145,91 | 26,26      | 27,22        | 6.41    |
| MAX      | 162,02 | 39,70      | 34,42        | 6.94    |

TABLE VI: Fusion performance comparison.

| Method           | Platform     | Embedded     | Time (ms)    | 3D mAP       |
|------------------|--------------|--------------|--------------|--------------|
| LPCF [26]        | Quadro M4000 | $\times$     | 90           | 62.71        |
| MV3D [32]        | Titan Xp     | $\times$     | 360          | 62.35        |
| F-PointNet [33]  | Titan Xp     | $\times$     | 170          | 70.39        |
| AVOD [31]        | Titan Xp     | $\times$     | 80           | 65.78        |
| AVOD-FPN [31]    | Titan Xp     | $\times$     | 100          | 71.88        |
| VoxelNet [40]    | Titan Xp     | $\times$     | 230          | 65.46        |
| UberATG-MMF [41] | N/D          | $\times$     | 80           | <b>76.75</b> |
| Beacons [34]     | NVIDIA Tx2   | $\checkmark$ | 200          | N/D          |
| this paper       | AGX Xavier   | $\checkmark$ | <b>29.43</b> | 66.40        |

## B. Calibration

To evaluate the precision of the proposed calibration method, we adopted the metric introduced in [22]. The miscal-



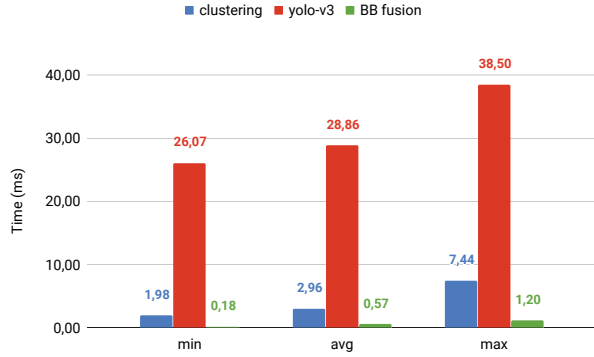


Fig. 8: Execution time of the whole method on the AGX Xavier.

ibration error has been computed both on calibration frames and on other frames. The results and the comparison with Velas [22] and LT [39] are shown in Table III.

### C. Detection

Table IV shows a performance comparison of the family of Yolo neural networks, including the original implementation<sup>4</sup>, the NVIDIA Deepstream Yolo plugin<sup>5</sup> and our solution, whose framework we called tkDNN. The tests have been carried out on an NVIDIA Tx2. As it can be noticed from the Table, our framework allows exploiting inference at full and half precision floating points, performing better than the other existing solutions.

### D. Fusion

To characterize the overall fusion framework, we performed a further set of experiments using the KITTI dataset [42]. Our WBLC algorithm was configured with parameters  $maxD = 0.3$ ,  $minCluster = 20$ ,  $maxN = 15$ ,  $W = 500$ , using the calibration, detection and fusion algorithms described in Section IV. The timing performances of the whole method on the AGX Xavier embedded platform are reported in Fig. 8. Note that most of the execution time is occupied by the detection algorithm. Table V shows the performance of the fusion framework on different computing platforms. Note that the Xavier platform (and Titan V) is able to natively run the detection network at 16-bit precision, leading to a smaller performance gap with the GTX 1060 than in the clustering case.

In a final set of experiments, we compared the execution time and accuracy of the state-of-the-art fusion methods available in the literature. Results are reported in Table VI, where the mean Average Precision (mAP) has been computed on the moderate car test set of the KITTI dataset [42]. Note that, contrarily to many of the compared solutions, we did not train our detection network with this dataset, as explained in Section IV. Despite this disadvantage, the precision of our

fusion framework is comparable to state-of-the-art solutions, but it is able to run in real-time on an embedded platform. The average execution time of the proposed framework on the embedded Xavier platform is three time faster than the best existing solutions running on desktop servers.

The results of the fusion technique are shown in Figure 9.

## VI. CONCLUSION

In this paper, we presented a fast, easy to implement and open-source method for 3D detection based on camera-LiDAR fusion. The framework is composed of a clustering algorithm customized for LiDAR data, a new camera-LiDAR alignment technique, and an optimized version of Yolo-v3 for the considered embedded platforms. The proposed framework has been validated on different datasets, including a new one specifically realized in an urban setting. The comparison against state-of-the-art solutions shows that our approach is able to conveniently run on embedded platforms in real-time, with limited performance degradation.

As a future work, we want to improve the accuracy of the method by predicting 3D bounding boxes directly from the camera, and merging them afterwards with LiDAR clusters, exploring the trade-off between the accuracy improvement and the additional computational complexity on the targeted embedded platforms.

## ACKNOWLEDGMENT

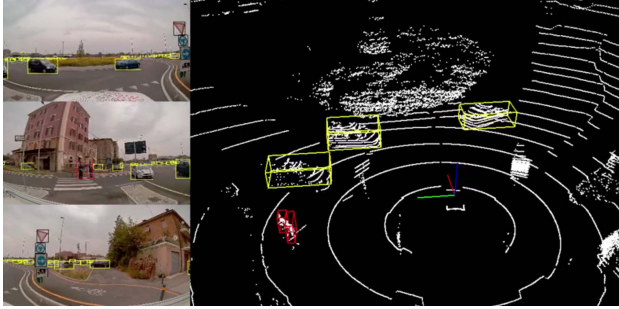
This work was supported by the Prystine Project, funded by Electronic Components and Systems for European Leadership Joint Undertaking (ECSEL JU) in collaboration with the European Union's H2020 Framework Programme and National Authorities, under grant agreement n° 783190.

## REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.
- [2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *European conference on computer vision*. Springer, 2016, pp. 21–37.
- [3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in neural information processing systems*, 2015, pp. 91–99.
- [5] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as points," *arXiv preprint arXiv:1904.07850*, 2019.
- [6] R. Xu and D. C. Wunsch, "Survey of clustering algorithms," 2005.
- [7] L. Rokach and O. Maimon, "Clustering methods," in *Data mining and knowledge discovery handbook*. Springer, 2005, pp. 321–352.
- [8] M. Ester, H.-P. Kriegel, J. Sander, X. Xu *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Kdd*, vol. 96, no. 34, 1996, pp. 226–231.
- [9] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [10] S. Lloyd, "Least squares quantization in pcm," *IEEE transactions on information theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [11] C. Böhm, R. Noll, C. Plant, and B. Wackersreuther, "Density-based clustering using graphics processors," in *Proceedings of the 18th ACM conference on Information and knowledge management*. ACM, 2009, pp. 661–670.

<sup>4</sup><https://github.com/pjreddie/darknet>

<sup>5</sup><https://developer.nvidia.com/deepstream-sdk>



(a) Fusion output. Cameras on the left, point cloud on the right.



(b) Coloured point cloud.

Fig. 9: Fusion outputs.

- [12] G. Andrade, G. Ramos, D. Madeira, R. Sachetto, R. Ferreira, and L. Rocha, "G-dbscan: A gpu accelerated algorithm for density-based clustering," *Procedia Computer Science*, vol. 18, pp. 369–378, 2013.
- [13] I. Bogoslavskyi and C. Stachniss, "Fast range image-based segmentation of sparse 3d laser scans for online operation," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 163–169.
- [14] P. M. Chu, S. Cho, Y. W. Park, and K. Cho, "Fast point cloud segmentation based on flood-fill algorithm," in *2017 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. IEEE, 2017, pp. 656–659.
- [15] P. Burger and H.-J. Wuensche, "Fast multi-pass 3d point segmentation based on a structured mesh graph for ground vehicles," in *2018 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2018, pp. 2150–2156.
- [16] P. Burger, B. Naujoks, and H.-J. Wuensche, "Fast dual decomposition based mesh-graph clustering for point clouds," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 1129–1135.
- [17] M. Wen, S. Cho, J. Chae, Y. Sung, and K. Cho, "Range image-based density-based spatial clustering of application with noise clustering method of three-dimensional point clouds," *International Journal of Advanced Robotic Systems*, vol. 15, no. 2, p. 1729881418762302, 2018.
- [18] Q. Zhang and R. Pless, "Extrinsic calibration of a camera and laser range finder (improves camera calibration)," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)(IEEE Cat. No. 04CH37566)*, vol. 3. IEEE, 2004, pp. 2301–2306.
- [19] Y. Park, S. Yun, C. Won, K. Cho, K. Um, and S. Sim, "Calibration between color camera and 3d lidar instruments with a polygonal planar board," *Sensors*, vol. 14, no. 3, pp. 5333–5353, 2014.
- [20] H. Alismail, L. D. Baker, and B. Browning, "Automatic calibration of a range sensor and camera system," in *2012 Second International Conference on 3D Imaging, Modeling, Processing, Visualization & Transmission*. IEEE, 2012, pp. 286–292.
- [21] C. Guindel, J. Beltrán, D. Martín, and F. García, "Automatic extrinsic calibration for lidar-stereo vehicle sensor setups," in *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2017, pp. 1–6.
- [22] M. Vel'as, M. Španěl, Z. Materna, and A. Herout, "Calibration of rgb camera with velodyne lidar," 2014.
- [23] N. Kanopoulos, N. Vasanthavada, and R. L. Baker, "Design of an image edge detection filter using the sobel operator," *IEEE Journal of solid-state circuits*, vol. 23, no. 2, pp. 358–367, 1988.
- [24] H. Yuen, J. Princen, J. Illingworth, and J. Kittler, "Comparative study of hough transform methods for circle finding," *Image and vision computing*, vol. 8, no. 1, pp. 71–77, 1990.
- [25] M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, 1981.
- [26] H. Wang, X. Lou, Y. Cai, Y. Li, and L. Chen, "Real-time vehicle detection algorithm based on vision and lidar point cloud fusion," *Journal of Sensors*, vol. 2019, 2019.
- [27] X. Chen, K. Kundu, Z. Zhang, H. Ma, S. Fidler, and R. Urtasun, "Monocular 3d object detection for autonomous driving," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2147–2156.
- [28] A. Mousavian, D. Anguelov, J. Flynn, and J. Kosecka, "3d bounding box estimation using deep learning and geometry," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 7074–7082.
- [29] M. Engelcke, D. Rao, D. Z. Wang, C. H. Tong, and I. Posner, "Vote3deep: Fast object detection in 3d point clouds using efficient convolutional neural networks," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 1355–1361.
- [30] B. Li, T. Zhang, and T. Xia, "Vehicle detection from 3d lidar using fully convolutional network," *arXiv preprint arXiv:1608.07916*, 2016.
- [31] J. Ku, M. Mozifian, J. Lee, A. Harakeh, and S. L. Waslander, "Joint 3d proposal generation and object detection from view aggregation," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–8.
- [32] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3d object detection network for autonomous driving," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1907–1915.
- [33] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, "Frustum pointnets for 3d object detection from rgb-d data," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 918–927.
- [34] P. Wei, L. Cagle, T. Reza, J. Ball, and J. Gafford, "Lidar and camera detection fusion in a real-time industrial multi-sensor collision avoidance system," *Electronics*, vol. 7, no. 6, p. 84, 2018.
- [35] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 7263–7271.
- [36] W. Ali, S. Abdelkarim, M. Zidan, M. Zahran, and A. El Sallab, "Yolo3d: End-to-end real-time 3d oriented object bounding box detection from lidar point cloud," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 0–0.
- [37] A. W. Fitzgibbon, R. B. Fisher *et al.*, *A buyer's guide to conic fitting*. University of Edinburgh, Department of Artificial Intelligence, 1996.
- [38] F. Yu, W. Xian, Y. Chen, F. Liu, M. Liao, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving video database with scalable annotation tooling," *arXiv preprint arXiv:1805.04687*, 2018.
- [39] J. Levinson and S. Thrun, "Automatic online calibration of cameras and lasers," in *Robotics: Science and Systems*, vol. 2, 2013.
- [40] Y. Zhou and O. Tuzel, "Voxelnet: End-to-end learning for point cloud based 3d object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4490–4499.
- [41] M. Liang, B. Yang, Y. Chen, R. Hu, and R. Urtasun, "Multi-task multi-sensor fusion for 3d object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 7345–7353.
- [42] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *International Journal of Robotics Research (IJRR)*, 2013.